

Radar de Editais

Prazo: 4 dias corridos | Entrega: repositório Git + defesa ao vivo (45 min)

1. Contexto

A **Engevia Consultoria e Projetos** (perfil no Anexo A) perde oportunidades de negócio por um motivo bobo: saem centenas de contratações públicas por dia no PNCP e ninguém na empresa tem tempo de ler. Quando alguém lê, já passou o prazo.

Você vai construir um sistema que ingere editais públicos, extrai os dados que importam, **valida o que extraiu** e ranqueia por aderência ao perfil da Engevia. O sócio vai olhar apenas os 10 melhores do dia.

Um falso positivo custa o tempo do sócio. Um falso negativo custa um contrato. Um dado errado apresentado com confiança custa a confiança dele no sistema inteiro, e esse é o pior dos três.

2. O que construir

Escopo mínimo. Tudo abaixo precisa rodar:

- Ingestão.** Colete cerca de 50 contratações reais da API pública de consulta do PNCP. A API é aberta e não exige credencial. Descobrir como ela funciona faz parte do desafio.
- Extração.** Para cada contratação, extraia 8 campos estruturados: objeto, modalidade, valor estimado, prazo de entrega/execução, data de abertura das propostas, exigência de atestado de capacidade técnica, exigências de habilitação e município/UF.
- Validação determinística.** O LLM não pode ser juiz final de nada. Toda informação extraída passa por uma camada de verificação que não usa IA: dígito verificador de CNPJ, coerência entre datas, valor extraído tem que existir literalmente no texto de origem, UF tem que ser uma das 27, e assim por diante. O que não passar na validação deve ser marcado como não confiável, e nunca descartado silenciosamente nem "corrigido" por adivinhação.
- Score de aderência.** Uma nota de 0 a 100 indicando o quanto aquela contratação combina com a Engevia. O score precisa ser **rastreável**: cada ponto somado ou subtraído aponta para o trecho do edital que o justifica. "Score 82" sem explicação é inútil para o sócio.
- Saída.** Uma **API FastAPI**, com no mínimo um endpoint `GET /oportunidades?min_score=70`. A API é obrigatória: não aceitamos CLI no lugar dela. Se quiser um comando de linha para uso próprio durante o desenvolvimento, tudo bem, mas o que será avaliado e o que rodaremos na defesa é a API.

Fora do escopo

Não queime tempo com isto, porque não conta ponto e pode contar contra:

- Autenticação, usuários, permissões
- Deploy, Docker, CI/CD
- Banco relacional com migrations (SQLite ou arquivos JSON em disco resolvem)

Diferencial: front-end consumindo a API

Opcional. Não compensa falha no escopo mínimo, mas conta a favor de quem chegar lá com o resto funcionando:

- Uma interface simples que consuma a sua própria API e mostre as oportunidades do dia, com filtro por score mínimo e a **justificativa de cada ponto** do score visível na tela, inclusive os campos marcados como não confiáveis.
- Stack livre: React, Vue, HTMX ou HTML estático servido pelo próprio FastAPI. Não avaliamos capricho visual; avaliamos se o sócio abre e entende em 30 segundos.

Front bonito sobre API frágil pontua menos do que API sólida sem front. A ordem importa.

Código inicial. Você recebe junto com este documento o arquivo `pncp_client.py`, um cliente HTTP para adiantar a etapa de ingestão. Trate-o como *ponto de partida*, não como referência: é código de terceiro que você está herdando. Audite antes de confiar.

3. Regras do jogo

Estas restrições existem porque são elas que tornam o desafio interessante. Não são burocracia.

3.1 Modelo fraco, obrigatoriamente

Você só pode usar modelos de LLM de tier gratuito ou local na execução do pipeline: camada gratuita do Gemini, Groq, ou algo rodando local via Ollama. **Nada de Claude Opus, GPT-5 ou similares fazendo o trabalho pesado de extração.**

Você pode, e deve, usar IA de ponta para te ajudar a programar. A restrição é sobre o que roda dentro do seu pipeline, não sobre como você escreve o código. Reporte o custo médio e a latência média por documento.

3.2 Qualidade medida, não declarada

Anote **à mão** um conjunto de referência com 15 contratações: você lê o edital, você preenche os 8 campos, você salva o gabarito. Depois entregue um harness que roda com `python eval.py` e imprime a acurácia por campo contra esse gabarito.

Sim, anotar 15 documentos na mão é chato. É também a única forma de você saber se o seu sistema funciona, e é a parte do desafio em que você vai descobrir o quanto o problema é mais difícil do que parecia.

3.3 O sistema não pode inventar

Campo que não foi possível extrair com confiança deve retornar nulo **com o motivo registrado**. Um sistema que devolve 6 campos corretos e 2 marcados como incertos vale mais, nesta avaliação, do que um que devolve 8 campos dos quais 2 estão errados sem aviso.

3.4 Robustez a entrada suja

Documento público real é bagunçado. Podemos enviar material adicional durante o desafio, sem aviso prévio sobre o que ele contém. Seu sistema precisa processar o que der, falhar de forma explícita no que não der, e nunca derrubar o pipeline inteiro por causa de um documento ruim.

3.5 AI_LOG.md obrigatório

Um arquivo registrando onde você usou IA, **o que ela errou**, como você percebeu o erro e o que fez a respeito. Queremos ver você usando IA como ferramenta, com senso crítico, não como oráculo. Este arquivo pesa na avaliação.

3.6 Histórico de commits

Commite ao longo dos 4 dias. Um repositório com dois commits gigantes no último dia não permite avaliar o seu processo, e o processo é metade do que estamos olhando.

4. Entregáveis

Item	O que precisa conter
README.md	Como rodar em menos de 5 minutos numa máquina limpa, incluindo como subir a API (e o front, se você tiver feito). Assuma que quem vai rodar não é você.
DECISOES.md	5 decisões técnicas que você tomou, as alternativas que descartou e por quê. Inclua pelo menos uma que você mudaria se tivesse mais tempo.
AI_LOG.md	Uso de IA, erros dela, como você percebeu, o que corrigiu.
eval.py + gabarito	Harness rodável e os 15 documentos anotados à mão.
RESULTADOS.md	Acurácia por campo, custo médio por documento, latência média, e uma seção honesta sobre o que não funcionou.
Código	O sistema em si, no repositório, com histórico de commits.

5. Como será avaliado

Peso	Critério
30%	Validação e tratamento de erro. A camada determinística existe e realmente pega erro do LLM? O sistema sinaliza incerteza em vez de inventar? Aguenta entrada ruim?
25%	Defesa ao vivo. Você entende o que construiu e consegue mexer nele na nossa frente.
20%	Evals e honestidade dos números. Você mediu de verdade e os números batem com a realidade?
15%	Clareza das decisões. DECISOES.md e AI_LOG.md mostram raciocínio próprio?
10%	Código e organização. Legível, estruturado, sem entulho.

O front-end é **bônus**: entra por cima desses 100%, como critério de desempate entre entregas equivalentes. Nunca no lugar de nenhum deles.

Repare no que não está na tabela: "funcionar 100%".

Um candidato que entrega 70% de acurácia, sabe exatamente por que os outros 30% falham e mostra os dados vale mais para nós do que um que afirma estar perfeito sem ter medido nada. Preferimos 70% medido a 100% prometido.

6. A defesa

No 5º dia, 45 minutos, com tela compartilhada:

- **15 min.** Você apresenta o que construiu.
- **15 min.** Abrimos arquivos do seu repositório e você explica o que está ali. Esperamos perguntas do tipo "por que assíncrono aqui?", "o que acontece se essa lista vier vazia?", "por que não fez de outro jeito?".
- **15 min. Alteração ao vivo.** Vamos pedir uma mudança pequena que você não previu. Você pode usar IA, desde que narre o que está fazendo e por quê.

7. Conselho prático

- **Dia 1.** Entenda a API do PNCP e olhe os dados brutos com os próprios olhos antes de escrever pipeline. Você vai economizar dois dias.
- **Dia 2.** Anote o gabarito. É chato, faça mesmo assim, e faça cedo. Anotar depois de já ter construído o sistema contamina o gabarito.
- **Dia 3.** Extração, validação e score.
- **Dia 4.** Evals, documentação, e o lote extra que você recebeu. Se sobrar tempo aqui, é onde entra o front-end.

Se travar em algo por mais de duas horas, mande mensagem. Saber pedir ajuda no momento certo é habilidade técnica, não fraqueza, e como você pergunta também diz muito.

Anexo A: Perfil da Engevia Consultoria e Projetos LTDA

Use este perfil como base para o score de aderência. Ele é propositalmente incompleto: algumas situações não estão cobertas e você vai precisar decidir sozinho como tratá-las. **Documente essas decisões.**

Atributo	Valor
Ramo	Engenharia consultiva e execução de obras de infraestrutura urbana
CNAE principal	7112-0/00 (Serviços de engenharia)
Sede	Brasília / DF
Atuação geográfica	DF, Goiás e Minas Gerais. Fora disso, só com margem alta.
Porte	22 funcionários, 6 engenheiros com CREA ativo, faturamento anual em torno de R\$ 8 milhões
Especialidades	Drenagem urbana, pavimentação asfáltica, projeto básico e executivo, gerenciamento e fiscalização de obras
Atestados de capacidade técnica	Pavimentação asfáltica até 12 km · Drenagem urbana até R\$ 4 milhões · Elaboração de projeto executivo de infraestrutura viária
Teto operacional	Consegue tocar contratos de até R\$ 5 milhões simultaneamente
Piso de interesse	Abaixo de R\$ 150 mil normalmente não compensa o custo de participar
Restrições financeiras	Capital de giro limitado. Contrato com prazo de pagamento acima de 90 dias é inviável
Não faz	Edificação vertical, obra elétrica de alta tensão, saneamento de grande porte (ETE/ETA)
Não possui	Certificação ISO 9001 · Filial fora do DF · Frota própria de equipamento pesado
Modalidades preferidas	Pregão eletrônico e concorrência. Evita dispensa e inexigibilidade.

Situações que o perfil não cobre

Alguns exemplos do que você vai encontrar e vai ter que decidir:

- Um contrato de R\$ 4,8 milhões em drenagem urbana, dentro do teto, mas encosta no limite do atestado. Pontua alto ou médio?
- Uma contratação em Anápolis/GO com exigência de sede no município. A Engevia não tem filial lá. Elimina ou desconta?
- Um objeto misto: pavimentação (faz) somada a construção de uma guarita (edificação, não faz). Como pontuar?
- Um edital em que o valor estimado é sigiloso. Como o score se comporta quando falta um campo relevante?

Não existe resposta única certa para nenhuma delas. Existe resposta *defendida* e resposta improvisada, e a diferença aparece na leitura.